

Controle digital

Trabalho Vivencial — Controle Digital com o Robô Baratinha

Professor: Prof. Msc. Paulo Cirillo Souza Barbosa

Introdução

Este trabalho vivencial consolida o ciclo completo de engenharia de controle: modelagem, discretização analítica, projeto de controle por alocação de polos e implementação física. O desafio é estabilizar e controlar o robô *Baratinha* partindo apenas do modelo dinâmico contínuo.

Dados e Regras Gerais

- **Planta Contínua (SISO):**

$$G(s) = \frac{32,4s + 3240}{(s - 2,16)(s + 2,31)}$$

- **Período de amostragem:** $T_s = 0,01$ s.
- **Controlador PID Digital (Forma Posicional):**

$$C(z) = K_p + K_i \frac{z}{z-1} + K_d \frac{z-1}{z} \quad (\text{Estrutura ideal para projeto})$$

- **Nota:** O trabalho é dividido em discretização/análise (Parte A), projeto analítico (Parte B), simulação (Parte C) e implementação física (Parte D).

Parte A — Discretização Analítica e Análise

Nesta etapa, vocês devem deduzir o modelo discreto $G(z)$ matematicamente, sem usar funções prontas de conversão (como `c2d`). Em seguida, devem validar seus cálculos via software.

1. **Expansão em Frações Parciais:** Considerando que o sistema digital utiliza um Segurador de Ordem Zero (ZOH), a relação de discretização é dada por:

$$G(z) = (1 - z^{-1}) \mathcal{Z} \left\{ \frac{G(s)}{s} \right\}$$

- (a) Realize a expansão de $\frac{G(s)}{s}$ em frações parciais:

$$\frac{G(s)}{s} = \frac{A}{s} + \frac{B}{s - 2,16} + \frac{C}{s + 2,31}$$

Encontre os valores de A , B e C .

- (b) Aplique a Transformada Z tabela-a-tabela em cada termo e simplifique a expressão final para obter $G(z)$ na forma de uma única fração racional em z .
2. **Verificação Computacional (Python):** Utilize a biblioteca `python-control` para validar os resultados obtidos no item anterior:
 - (a) Utilize a função `control.pole()` para listar os polos de $G(s)$ e do $G(z)$ calculado. O sistema é estável em malha aberta? Justifique com base na posição dos polos no plano s e no plano z .
 - (b) Calcule o ganho DC da planta discretizada utilizando a função `control.dcgain()`.

Parte B — Projeto por Alocação de Polos (Analítico)

O objetivo é projetar os ganhos K_p, K_i, K_d para que a malha fechada se comporte conforme especificações rígidas.

1. **Requisitos de Desempenho (Projeto Reverso):** O sistema em malha fechada deve atender a:

- **Sobressinal (M_p):** $\leq 30\%$
- **Tempo de acomodação (t_s , 2%):** $< 3,0$ s

Determine analiticamente a região válida de ζ (amortecimento) e ω_n (frequência natural) que satisfaz essas desigualdades. Utilize as fórmulas de aproximação de 2ª ordem para obter as restrições:

- **Restrição de ζ (Sobressinal):** $\zeta \geq \frac{|\ln(M_p)|}{\sqrt{\pi^2 + (\ln(M_p))^2}}$
- **Restrição de ω_n (Tempo de Acomodação):** $\omega_n > \frac{4}{\zeta t_s}$

2. **Escolha dos Polos de Malha Fechada:** Devido à dinâmica da planta (2ª ordem) e do PID (que insere polos em $z = 1$ e $z = 0$), a equação característica será de **4ª ordem**. Porém, projetaremos focando nos 3 polos principais controláveis pelos ganhos K_p, K_i, K_d .

- Escolha um par de polos dominantes $s_{1,2}$ (dentro da região calculada acima) e mapeie para z ($z_{1,2} = e^{s_{1,2}T_s}$).
- Escolha um terceiro polo auxiliar z_3 (real) que seja mais rápido que os dominantes para não interferir na dinâmica principal. (Recomendação: escolher z_3 com α de 5 a 10 vezes maior que $\zeta\omega_n$).

3. **Cálculo dos Ganhos (Matching de Coeficientes):** Para encontrar os ganhos do PID, siga este roteiro:

- (a) Escreva a equação característica da malha fechada simbólica. Lembre-se que $C(z)G(z)$ deve ser expresso com um denominador comum.

$$1 + C(z)G(z) = 0 \quad \implies \quad \text{Denominador} + \text{Numerador} = 0$$

- (b) Simplifique a expressão, eliminando o denominador e agrupando os termos, até obter um polinômio da forma:

$$z^3 + a_2(K)z^2 + a_1(K)z + a_0(K) = 0$$

Onde os coeficientes a_n serão funções lineares (combinações) de K_p, K_i, K_d e dos parâmetros da planta.

- (c) Igualar os coeficientes a_n aos coeficientes desejados d_n :

$$\begin{cases} a_2(K_p, K_i, K_d) = d_2 \\ a_1(K_p, K_i, K_d) = d_1 \\ a_0(K_p, K_i, K_d) = d_0 \end{cases}$$

- (d) **Solução por Sistema Linear:** Resolva o sistema montando a matriz de coeficientes A (que dependerá apenas dos parâmetros da planta) multiplicada pelo vetor de ganhos $\mathbf{x} = [K_p, K_i, K_d]^T$ e igualando ao vetor $\mathbf{b}$ (que dependerá dos coeficientes desejados d_n e dos termos constantes da planta). Utilize `numpy.linalg.solve` ou inversão de matriz para encontrar os valores finais de K_p, K_i, K_d .

Exemplo Ilustrativo do Sistema Linear (Não use os valores deste exemplo): Suponha que, após a expansão simbólica, o polinômio da malha fechada seja:

$$z^3 + (1.5 + 0.1K_p + 0.2K_i)z^2 + (-0.8 - 0.5K_p + 0.1K_d)z + (0.01 + 0.3K_d)$$

E o polinômio desejado seja: $z^3 + \mathbf{1.0}z^2 + \mathbf{0.2}z + \mathbf{0.05}$.

A igualdade termo a termo leva ao seguinte sistema $\mathbf{Ax} = \mathbf{b}$:

$$\begin{cases} 0.1K_p + 0.2K_i + 0.0K_d &= 1.0 - 1.5 \\ -0.5K_p + 0.0K_i + 0.1K_d &= 0.2 - (-0.8) \\ 0.0K_p + 0.0K_i + 0.3K_d &= 0.05 - 0.01 \end{cases}$$

Que é resolvido na forma matricial:

$$\underbrace{\begin{bmatrix} 0.1 & 0.2 & 0.0 \\ -0.5 & 0.0 & 0.1 \\ 0.0 & 0.0 & 0.3 \end{bmatrix}}_A \underbrace{\begin{bmatrix} K_p \\ K_i \\ K_d \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} -0.5 \\ 1.0 \\ 0.04 \end{bmatrix}}_{\mathbf{b}}$$

A solução é dada por $\mathbf{x} = A^{-1}\mathbf{b}$.

4. **Solução por Sistema Linear:** Resolva o sistema montando a matriz de coeficientes A (que dependerá apenas dos parâmetros da planta) multiplicada pelo vetor de ganhos $\mathbf{x} = [K_p, K_i, K_d]^T$ e igualando ao vetor $\mathbf{b}$ (que dependerá dos coeficientes desejados d_n). Utilize `numpy.linalg.solve` ou inversão de matriz para encontrar os valores finais de K_p, K_i, K_d .
5. **Verificação de Estabilidade (Jury):** Aplique o Critério de Jury completo no polinômio obtido com os ganhos calculados para provar matematicamente a estabilidade.

Parte C — Simulação e Validação

Utilize Python (`python-control`) para validar o projeto antes de ir para o robô.

1. Plote o **Lugar das Raízes** (Root Locus) mostrando os polos projetados.
2. Simule a **Resposta ao Degrau** unitário em malha fechada. Verifique se o M_p e t_s foram atendidos.
3. Plote o **Sinal de Controle** $u[k]$. Se o sinal exceder limites físicos (ex: saturação dos motores), discuta como isso afetaria o desempenho real.

Parte D — Implementação Física

Nesta etapa final, os ganhos calculados na Parte B devem ser embarcados no robô real para verificar o funcionamento do controle em um cenário prático.

1. **Programação do Firmware:** Implemente a lei de controle PID discreta em C++ no microcontrolador do robô.
 - Insira os valores de K_p, K_i, K_d obtidos no projeto analítico.
 - **Tratamento de Ruído:** Implemente o termo derivativo com filtro (conforme estrutura sugerida em aula) para evitar que o ruído do sensor cause vibrações nos motores.
 - **Saturação e Anti-Windup:** Garanta que o sinal de saída respeite os limites do PWM e impeça o crescimento excessivo do termo integral (windup) quando o motor atingir sua velocidade máxima.
 - **Segurança:** Adicione uma condicional para desligar os motores caso a distância medida seja inferior a um limite mínimo de segurança (3 cm).
2. **Validação Funcional:** Realize o teste de bancada ou pista:

- Defina um *setpoint* fixo (ex: manter 10 cm de distância de um obstáculo).
- Aplique perturbações manuais leves (deslocar o robô para frente ou para trás) e observe visualmente a capacidade de recuperação e estabilização.
- O critério de sucesso é o robô conseguir manter a distância alvo sem colidir com o obstáculo e sem oscilações permanentes.

Entregáveis

1. **Memória de Cálculo (PDF):** Contendo a discretização passo a passo (frações parciais), a dedução do sistema linear de ganhos e a tabela de Jury.
2. **Código Python:** Scripts de simulação.
3. **Vídeo/Dados:** Comprovação do funcionamento do robô.

Repositório do robô

https://github.com/joaoaugustocz/Baratinha_ControleDigital